Road Sign Detection

1. Course Content

Start the YOLO traffic sign recognition program. The model provided in the tutorial only recognizes the road signs included in the sandbox package. If you need to recognize other road signs, you need to train them yourself.

2. Road Sign Detection

Road sign detection source code path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

3. Program Startup

3.1 Startup Command

- Parameter Configuration

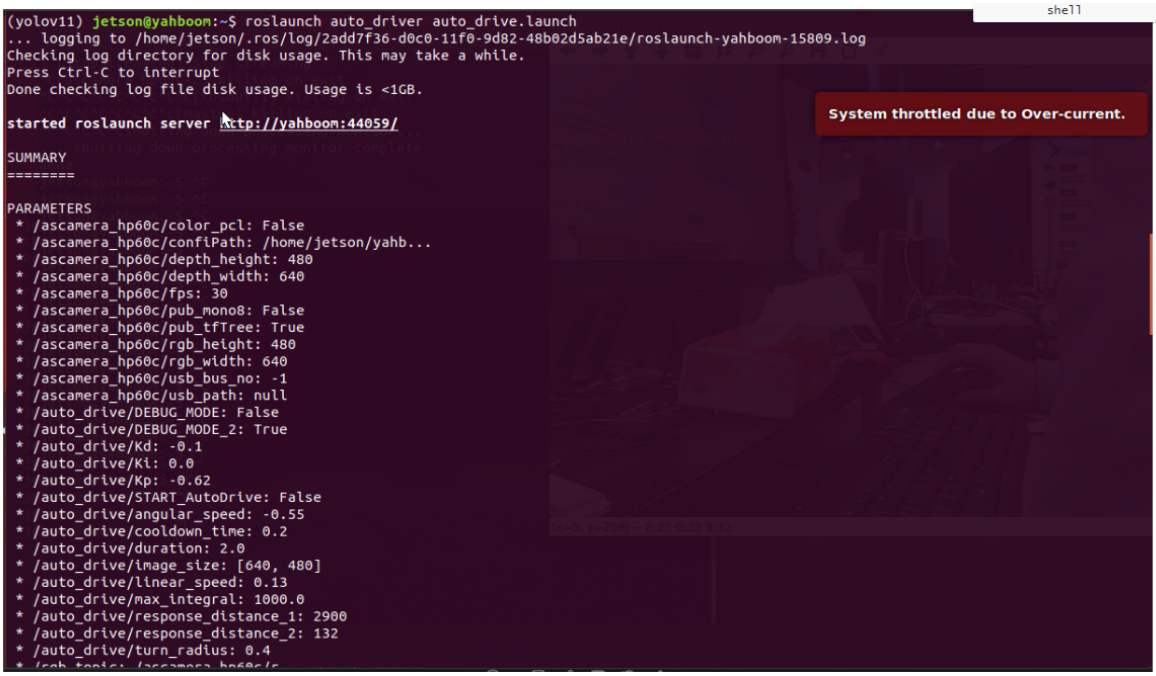
Parameter path:
`/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml`

```
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: False
DEBUG_MODE_2: True
START_AutoDrive: True
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

After modifying parameters, restart the launch file to use the modified parameters.

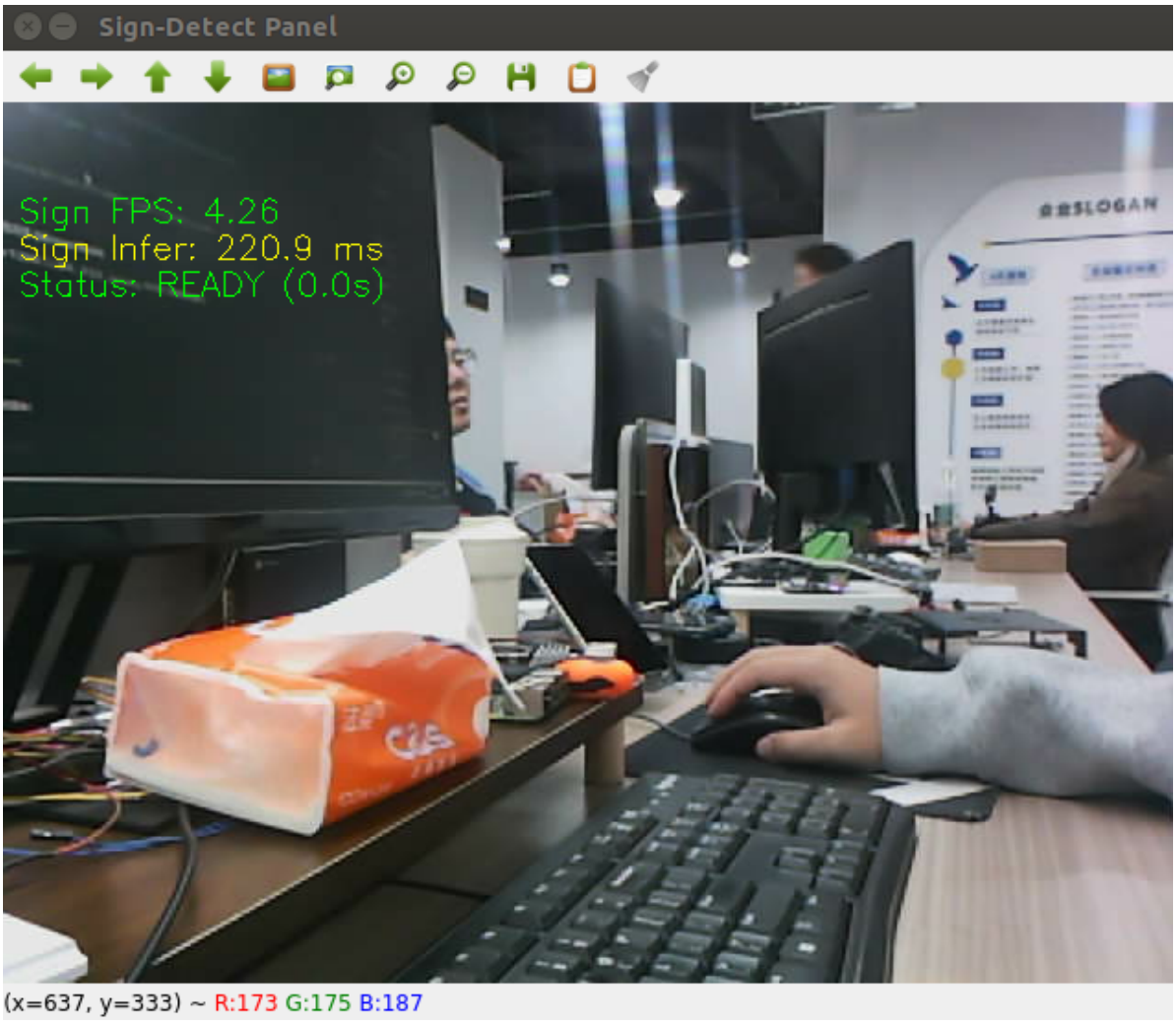
- Start camera + chassis + autonomous driving node (need to run in virtual environment terminal)

```
roslaunch auto_driver auto_drive.launch
```

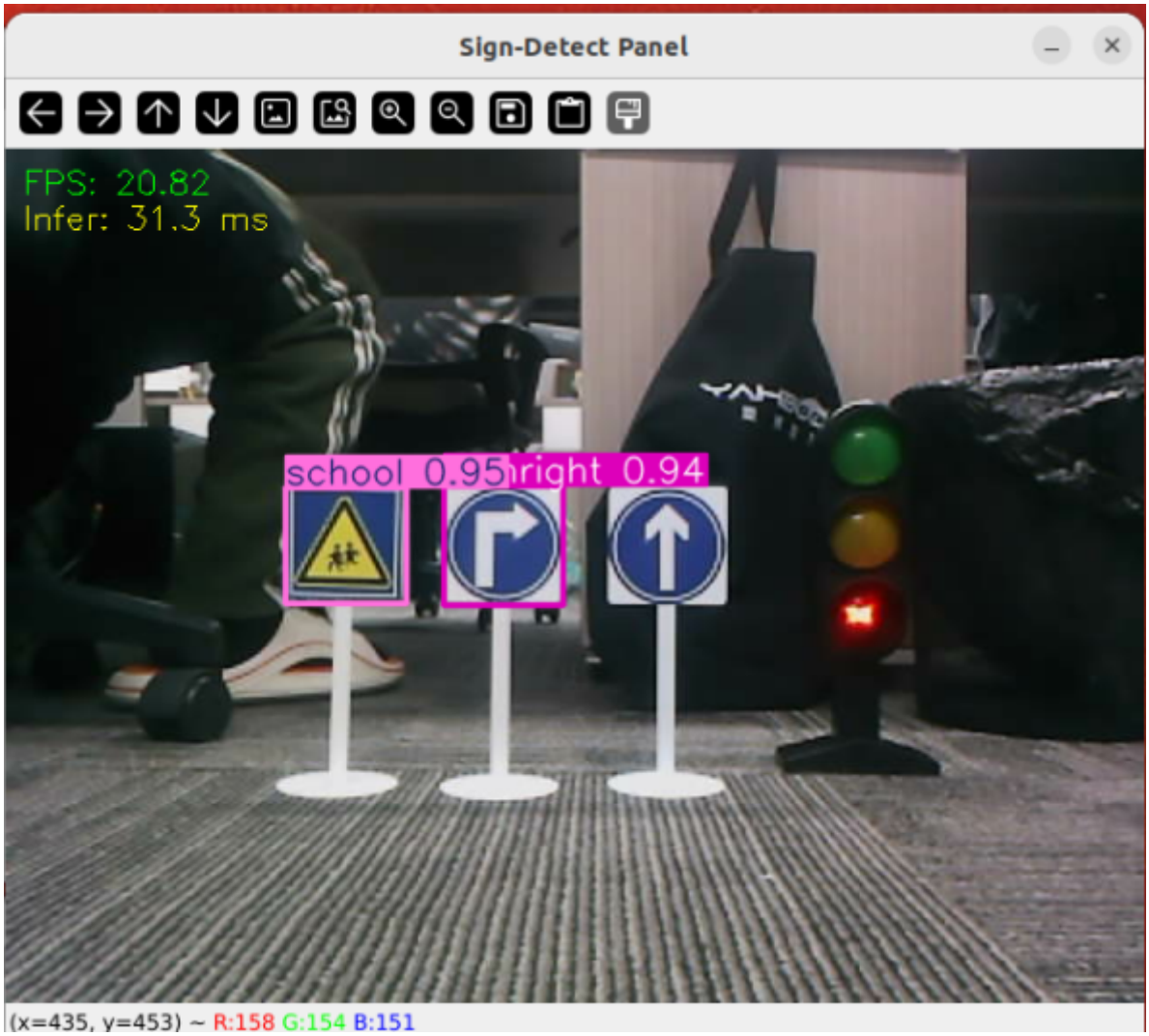


After successful startup, the display will not move to test road sign recognition. (If you want it to move, change the parameter START_AutoDrive in auto_drive_node.yaml to True)

After successful startup, a visualization interface will appear.



Place road signs in the center of the screen to recognize the correct sign names. The factory program defaults to recognizing only two road signs at the same time to avoid false recognition during autonomous driving.



You can adjust the YOLO recognition effect by modifying the YAML file in this path.

Parameter path: `/home/jetson/yahboomcar_auto/src/auto_driver/config/yolo_param.yaml`

auto_drive.py | yolo_param.yaml | auto_drive_node.yaml | midline_points.yaml

yahboomcar_auto > src > auto_driver > config > yolo_param.yaml

```yaml
#路标检测推理参数
sign_detection:
  model_path: '/home/jetson/yahboomcar_auto/src/sign_model.engine'
  conf: 0.92   #设置检测的最小置信度阈值。 将忽略置信度低于此阈值的检测到的对象。 调整此值有助于减少误报。
  iou: 0.7     #用于非极大值抑制 (NMS) 的重叠阈值。较低的值会通过消除重叠的框来减少检测结果
  rect: True     #如果启用，则对图像较短的一边进行最小填充，直到可以被步长整除，以提高推理速度。
  half: True      #如果启用，则使用半精度推理，可以加快在支持的 GPU 上的模型推理速度，同时对准确性的影响极小。
  device: "cuda:0"       #推理设备
  batch : 50          #批处理大小，更大的批处理大小可以提供更高的吞吐量，从而缩短推理所需的总时间。
  max_det : 2          #最多检测的物体数
  augment : False      #启用测试时增强 (TTA) 进行预测，可能会提高检测的鲁棒性，但会降低推理速度。
  verbose : False       #是否在终端输出推理的详细信息。
  classes : [0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13]

line_detection:
#车道线检测推理参数
  model_path: '/home/jetson/yahboomcar_auto/src/lane_V6.engine'
  conf: 0.5       #设置检测的最小置信度阈值。 将忽略置信度低于此阈值的检测到的对象。 调整此值有助于减少误报。
  iou: 0.6        #用于非极大值抑制 (NMS) 的重叠阈值。较低的值会通过消除重叠的框来减少检测结果
  rect: True      #如果启用，则对图像较短的一边进行最小填充，直到可以被步长整除，以提高推理速度。
  half: True       #如果启用，则使用半精度推理，可以加快在支持的 GPU 上的模型推理速度，同时对准确性的影响极小。
  device: "cuda:0"      #推理设备
  batch : 50         #批处理大小，更大的批处理大小可以提供更高的吞吐量，从而缩短推理所需的总时间。
  max_det : 4        #最多检测的物体数
  augment : False      #启用测试时增强 (TTA) 进行预测，可能会提高检测的鲁棒性，但会降低推理速度。
  verbose : False      #是否在终端输出推理的详细信息。
```

Where max_det: 2 indicates the maximum number of objects to detect, conf: 0.92 indicates the minimum confidence threshold for detection. Detected objects with confidence below this threshold will be ignored. Adjusting this value helps reduce false positives. After changing max_det: 4 and running the program, you can see four road signs boxed in the image.

For subsequent comprehensive autonomous driving applications, it is recommended to use 2 maximum targets here.

4. Source Code Analysis

4.1, auto_drive.py

Program source code path:
`/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_drive.py`

A ROS-based YOLO object detection node specifically designed for real-time traffic sign detection. The node receives camera images, uses YOLO model for object detection, and publishes detection results for other nodes to use.

Multi-threaded Processing Architecture

- **Processing Thread**: Specifically performs YOLO inference and result processing
- **Daemon Thread**: Ensures threads terminate correctly when program exits
- **Ignore List**: Filters sign types that don't need processing

```
 def _start_processing_threads(self):
        '''Start processing threads'''
        # Sign detection thread
        self.sign_thread = threading.Thread(target=self._sign_detection_worker)
        self.sign_thread.daemon = True
        self.sign_thread.start()

        # Event handling thread
        self.event_thread = threading.Thread(target=self._event_handling_worker)
        self.event_thread.daemon = True
        self.event_thread.start()
```

Image Callback Function

- Use `_imgmsg_to_cv2` to convert ROS image messages to OpenCV format
- Create image copy to avoid data race
- Put image into processing queue for detection thread to use

```
def image_callback(self, msg):
        '''Image callback function'''
        try:
            self.cv_image = self._imgmsg_to_cv2(msg)
        except CvBridgeError as e:
            rospy.logerr("CvBridge error: {}".format(e))
            return
```

Object Detection Main Loop

```
def _sign_detection_worker(self):
        '''Sign detection worker thread'''
        sign_pub_counter = 0

        while not rospy.is_shutdown() and not self._shutdown_flag:
            if self.cv_image is None:
                time.sleep(0.01)
                continue

            # Check cooldown status
            current_time = time.time()
            if self.sign_cooldown:
                if current_time - self.last_sign_time >= self.cooldown_duration:
                    self.sign_cooldown = False
                    rospy.loginfo("Sign detection cooldown ended")
                else:
                    # During cooldown period, skip sign detection
                    time.sleep(0.01)
                    continue
```

Result Parsing and Publishing

- **Frequency Control**: Publish once every 5 frames to balance real-time performance and system load
- **Data Type Conversion**: Ensure data format meets message type requirements

```
try:
```

```
    frame = self.cv_image.copy()
    infer_start = time.time()

    # Perform sign detection
    results = self.sign_detect.get_infer_result(frame, True)
    annotated_frame = frame.copy()

    # Record whether detected sign requires cooldown
    detected_sign_requires_cooldown = False

    for res in results:
        if res.boxes is not None:
            annotated_frame = res.plot()
            boxes = res.boxes
            for i in range(len(boxes)):
                box_coords = boxes.xyxy[i].tolist()
                class_name = res.names[int(boxes.cls[i].item())]
                confidence = boxes.conf[i].item()

                # Check if it's an ignored sign
                if class_name in self.ignored_signs:
                    # rospy.loginfo(f"Ignoring sign: {class_name}")
                    continue  # Skip ignored signs

                # Publish detection results
                if sign_pub_counter % 5 == 0:
                    detection_msg = DetectionObject()
                    detection_msg.class_name = class_name
                    detection_msg.confidence = confidence
                    detection_msg.xmin = float(box_coords[0])
                    detection_msg.ymin = float(box_coords[1])
                    detection_msg.xmax = float(box_coords[2])
                    detection_msg.ymax = float(box_coords[3])
                    self.sign_publisher.publish(detection_msg)

                    # Add detection result to event queue (ignored signs are not
added)
                    if not self.action_running:
                        object_distance =
self._get_distance(float(box_coords[1]))
                        obj = Object(class_name, confidence, object_distance)
                        try:
                            self.event_queue.put_nowait(obj)
                        except queue.Full:
                            pass

                        # Set cooldown flag (all signs except ignored ones
trigger cooldown)
                        detected_sign_requires_cooldown = True

                sign_pub_counter += 1

    # If detected sign requires cooldown, start cooldown
    if detected_sign_requires_cooldown:
        self.sign_cooldown = True
        self.last_sign_time = current_time
        rospy.loginfo(f"Sign detected, starting {self.cooldown_duration}s
cooldown")
```

```
except Exception as e:
    rospy.logerr("Error in sign detection: %s", str(e))
    time.sleep(0.01)
```